

AI assignment

1) Explain the current state of the art in Artificial Intelligence with suitable real world applications.

A) The current state of Art in AI

* The field of AI has evolved from simple rule based systems to Generative and Agentic systems. At present, the "state of the Art" (SOTA) is defined by three main characteristics:

1. Key Technologies

* Large Language Models : These are the backbone of modern AI. They use transformer architectures to understand and generate human like text by predicting the next token in a sequence.

* Multimodality : Modern AI is no longer restricted to just text. It can "see", "hear", and "speak" simultaneously. This allows the AI to process the world more like a human does.

* Autonomous Agents : Instead of just

answering questions, AI can now performs tasks. An AI agent can plan a trip, book a flight, and update your calendar without you doing each step manually.

2. Real world Applications

- * Healthcare : AI is used for Automated diagnostics. It can scan Xrays or MRI's to detect tumors or fractures with higher Precision and speed than manual observation.
- * Autonomous vehicles : self driving cars use computer vision and Reinforcement Learning to navigate traffic, recognize pedestrians, and make split-second safety decisions.
- * Education & Creative Arts : Tools like chatgpt or Gemini provide personalized tutoring by explaining complex topics in simple ways, while also assisting in the creation of high quality digital art and videos based on user descriptions.

3. Current Limitations

Despite its progress, state of the Art in AI still faces challenges like "Hallucinations and high computational costs. Engineering

focus is now shifting toward making these models smaller, faster, and more ethical.

2a) Explain the concept of Rationality in Artificial Intelligence with a Vacuum cleaner Example?

A The concept of Rationality in AI

* A Rational agent is one that acts so as to achieve the best outcome or when there is uncertainty, the best expected outcome. In AI, rationality is defined by four factors:

1. performance Measure: The criteria used to define success.
2. Prior Knowledge: The agent already knows about its environment.
3. Percept sequence: Everything the agent has sensed up to that point.
4. Action capability: The set of actions the agent can actually perform.

The vacuum cleaner Example:

* Imagine a robotic vacuum cleaner in a room with two squares (A & B):

→ The Actions :- The agent senses if a square is [Dirty] or [clean] and identifies its current location. It can move [left], [Right], [suck] dirt and [No-op] nothing done.

→ The performance Measure :- points are awarded for every clean square over a specific time period. (Ex :- +10 for cleaning square, -1 for every move to conserve battery).

* A vacuum cleaner is considered rational if it follows a logic that maximizes its score based on its sensor data :

→ If square A is Dirty, the rational action is to suck.

→ If square A is clean, the rational action is to move to square B to look for more dirt.

* Rationality is not omniscience. It is not irrational for not knowing there is dirt there. It is rational as long as it makes the best choice given its limited information.

2b) what is a multi agent environment? How

is it different from a single agent environment?

A) Multi Agent Environment

* A multi agent environment is one where multiple autonomous agents coexist and interact within the same space. In this setup, the success of an agent depends not just on its own actions and the environment, but also on the behaviour of the other agents. These interactions can be:

1. competitive: The basic example is about two robots playing chess where one's win is the other's loss.
2. cooperative: The basic example is about two robots working together to lift a heavy crate.

* The key difference of both environments lies in interference and strategy.

→ In single Agent Environment, the environment is essentially passive. The agent only needs to worry about its own actions and static obstacles.

→ In multi agent environment, the environment is essentially active or dynamic. The agent only model the goals and likely actions of other agents to make a rational decision.

3) Explain the working of simple reflex agents and model based reflex agent using neat schematic diagrams?

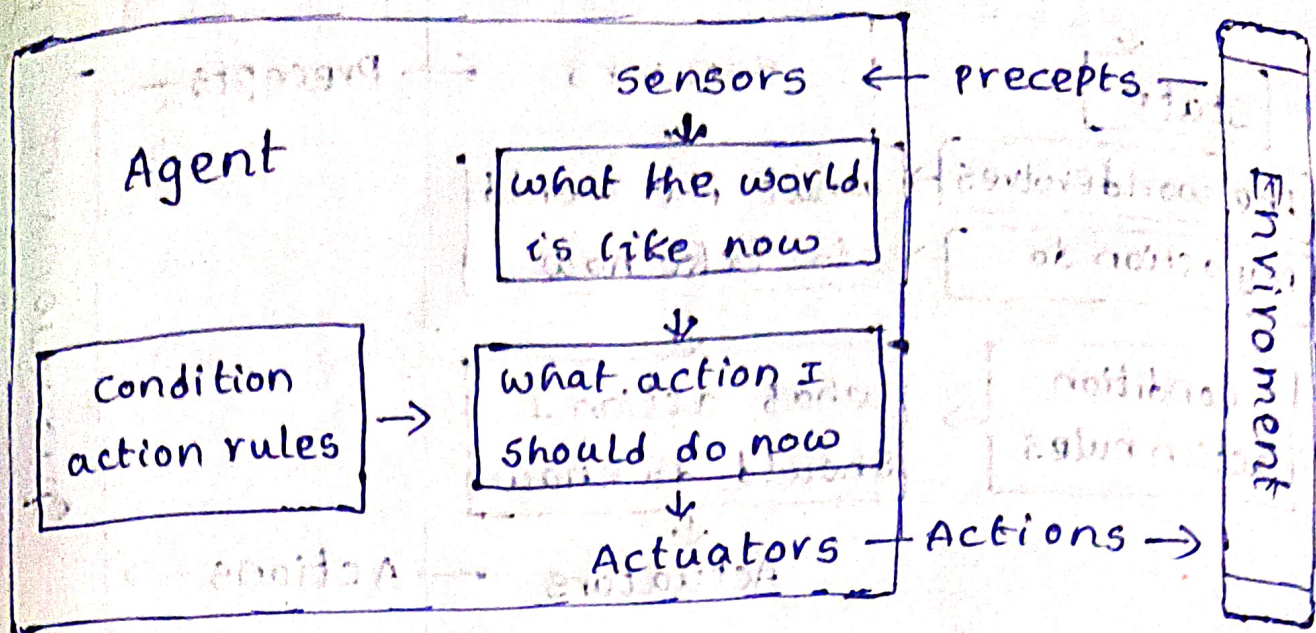
A) Simple reflex agent :-

* A simple reflex agent is the most basic type of AI. It works on the condition Action rule. It ignores the rest of the precept history and acts only on the current state of the environment.

working :- It maps the current percept directly to an action. It only works effectively in environments that are fully observable.

Example :- A vacuum cleaner that simply sucks if it senses dirty in the current square.

schematic diagram :-



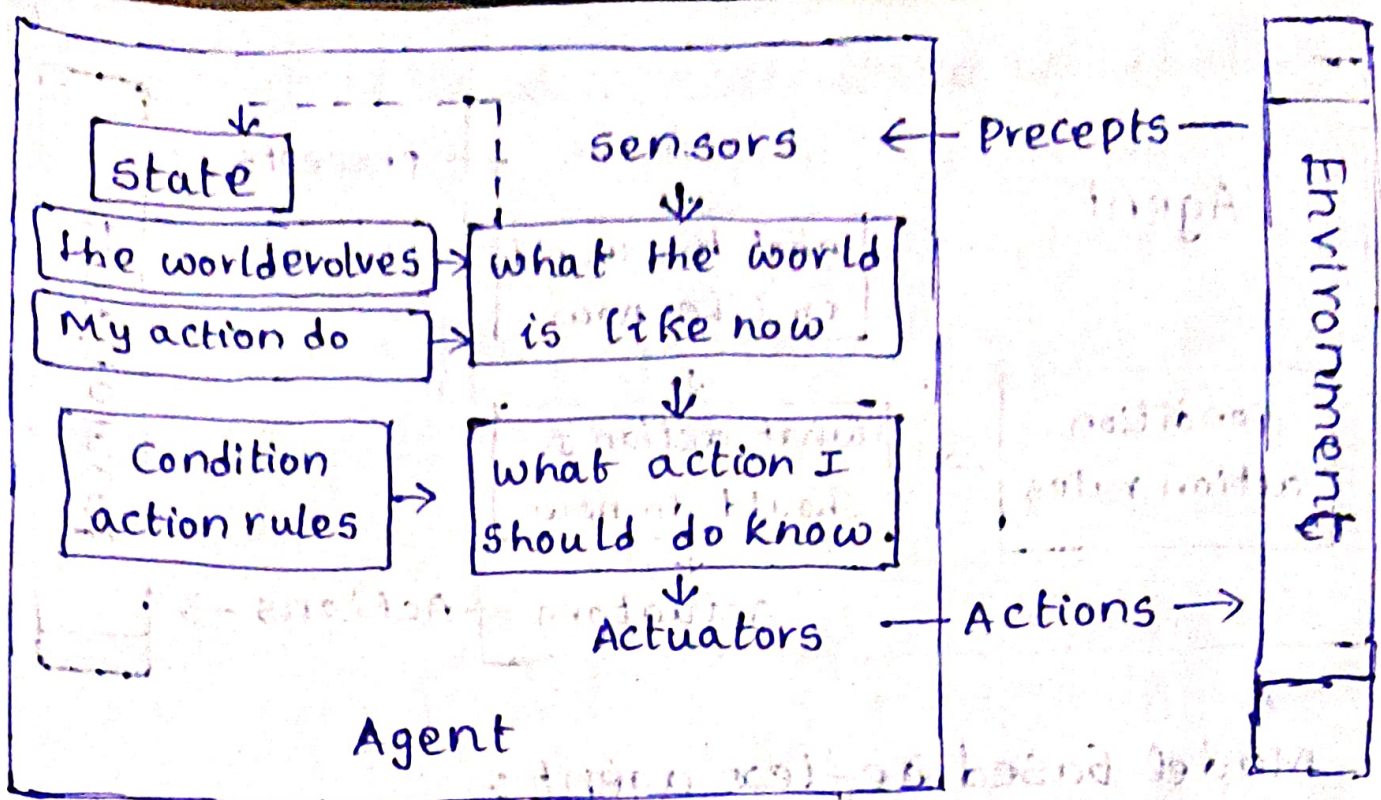
Model Based reflex agent :-

* A model based reflex agent is designed to handle partially observable environments. It maintains an internal model. A hidden state that tracks parts of the world it cannot see right now.

working :- It combines the current percept with its internal state. It keeps track of how the world evolves independently.

Example :- An Automated car when it changes lanes, it must remember that there was a car behind it 2 seconds ago even if that car is currently in its blind spot.

schematic diagram :-



4) Explain the following uninformed search strategies with suitable examples:

- i) uniform cost search
- ii) Depth limited search
- iii) Bidirectional search.

A)

i) Uniform Cost Search

Uniform cost search is used when step costs vary. It expands the node with the lowest path cost from the root. It is optimal because it guarantees the cheapest path.

Program :-


```

def UCS (graph, start, goal):
    queue = [[0, start, [start]]]
    visited = []
    while queue:
        queue.sort()
        cost, node, path = queue.pop(0)
        if node == goal:
            return path, cost
        if node not in visited:
            visited.append(node)
            for neighbor, weight in graph[node]:
                new_path = path + [neighbor]
                queue.append([cost + weight, neighbor, new_path])
    return "path not found"

```

ii) Depth limited search

DLS is a variation of Depth first search that stops at a predefined limit L . This prevents the algorithm from getting lost in infinite

Paths or deep branches in large state spaces.

Program :-

```
def DLS (node, goal, limit, path):  
    Path.append (node)  
    if node == goal:  
        return path  
    if limit <= 0:  
        return None  
    for neighbor in graph.get (node, []):  
        result = DLS (neighbor, goal,  
            limit-1, list (path))  
        if result:  
            return result  
    return None
```

iii) Bidirectional search

This strategy runs two simultaneous searches one forward from the initial state, and one backward from the goal state. The search stops when the two "frontiers" meet in the middle.

program :-

```
def bidirectional (start, goal, graph):  
    fwd_queue = [start]  
    bwd_queue = [goal]  
    fwd_visited = {start}  
    bwd_visited = {goal}  
  
    while fwd_queue and bwd_queue:  
        curr_f = fwd_queue.pop(0)  
        for n in graph.get(curr_f, []):  
            if n in bwd_visited:  
                return f"meeting point: {n}"  
            if n not in fwd_visited:  
                fwd_visited.add(n)  
                fwd_queue.append(n)  
  
        curr_b = bwd_queue.pop(0)  
        for n in graph.get(curr_b, []):  
            if n in fwd_visited:  
                return f"meeting point: {n}"  
            if n not in bwd_visited:  
                bwd_visited.add(n)  
                bwd_queue.append(n)  
  
    return "No path"
```


5) Apply memory bounded search strategies (RBFS, and SMA*) to solve Romania map Problem of finding a route from Arad to Bucharest. Explain step by step how each algorithm explores the search space under memory limitations?

A)

Recursive Best First Search (RBFS)

RBFS is a forgetful but smart version of A*. It uses a recursive approach and keeps track of the f -value of the best alternative path available from any ancestor.

Step by step Exploration:

1. start at Arad: It looks at neighbors (Sibiu, Zerind, Timisoara). It picks the best one (Sibiu) but remembers the f value of the second best option (Timisoara).
2. The Limit check: As it goes deeper into Sibiu's children (Fagaras, Rimnicu, Iilcea) it constantly compares the current path's cost against the best alternative it left behind at Arad.

3. Backtrack : If the path through sibiu becomes more expensive than the alternative at arad (exceeds the limit), RBFS deletes the sibiu subtree from memory to save space.

4. The memory trick : Before deleting, it saves the best f -value it found in that subtree. If it needs to come back later, it knows exactly how bad or good that path without recalculating everything.

simplified Memory Bounded A^* (SMA *)

SMA * is a organized version. It uses all available memory to store nodes, and only starts deleting them when the memory is completely full.

step by step Exploration :

1. Fill the Buffer : It acts like A^* , expanding nodes (Arad, sibiu, Rimnicu, Velcea) and storing them in a queue until the memory limit is reached.

2. Dropping the worst node : once memory is full if SMA * needs to add a new node,

it drops the leaf node with the highest f value.

3. Backing up values: Like RBFs, it does not just delete a node, it updates the Parent with the f value of its best forgotten child. This way, the parent remembers that a path existed there.

4. Re-Expansion: If all others turn out to be worse than the forgotten path SMA*, will delete a currently held node to regenerate the path it previously dropped.

6) Explain the process of the simulated annealing algorithm with a suitable example.

A) The process of simulated annealing algorithm

The algorithm works by allowing bad moves early on to avoid getting stuck in local traps. The process follows these steps:

1. Initialize: start with a random

initial solution (s) and a high initial temperature (T)

2. Iterate : At each step, pick a random neighbor (s') of the current state.
3. calculate change (ΔE): compute the difference in energy between the two states.

$$\Delta E = \text{value}(s') - \text{value}(s)$$

4. Acceptance criteria: If s' is better then accept it immediately as the new current state. if s' is worse then discard it.
5. Cooling: Gradually decrease the temperature (T) according to a cooling schedule (Ex., $T = T \times 0.95$)
6. Repeat: continue until the temperature reaches near zero.

The Mountain Climber in a fog Example:

If the climber only ever moves upward, they might reach the top of a small hill and get stuck there, thinking it's the highest point because every direction from

there is down

The solution :-

High temperature : The climber is allowed to take large, random steps even jumping down into valleys. This helps them leave the small hill and find the base of a much taller mountain.

Cooling Down : As the fog clears, the climber becomes more cautious. They stop jumping down and focus only on moving upwards.

Frozen state : Eventually the climber reaches the highest summit and stops, as they no longer take downward risks.

K.E.K